

2. Functions

The first organizational tool programmers use in Python is the *function*. As in other programming languages, functions enable you to break large programs into smaller, simpler pieces. They improve readability and make code more approachable. They allow for reuse and refactoring.

Functions in Python have a variety of extra features that make the programmer's life easier. Some are similar to capabilities in other programming languages, but many are unique to Python. These extras can make a function's purpose more obvious. They can eliminate noise and clarify the intention of callers. They can significantly reduce subtle bugs that are difficult to find.

Item 14: Prefer Exceptions to Returning None

When writing utility functions, there's a draw for Python programmers to give special meaning to the return value of `None`. It seems to make sense in some cases. For example, say you want a helper function that divides one number by another. In the case of dividing by zero, returning `None` seems natural because the result is undefined.

```
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return None
```

Code using this function can interpret the return value accordingly.

```
result = divide(x, y)
if result is None:
    print('Invalid inputs')
```

What happens when the numerator is zero? That will cause the return value to also be zero (if the denominator is non-zero). This can cause problems when you evaluate the result in a condition like an `if` statement. You may accidentally look for any `False` equivalent value to indicate errors instead of only looking for `None` (see [Item 4: “Write Helper Functions Instead of Complex Expressions”](#) for a similar situation).

[Click here to view code image](#)

```
x, y = 0, 5
result = divide(x, y)
if not result:
    print('Invalid inputs') # This is wrong!
```

This is a common mistake in Python code when `None` has special meaning. This is why returning `None` from a function is error prone. There are two ways to reduce the chance of such errors.

The first way is to split the return value into a two-tuple. The first part of the tuple indicates that the operation was a success or failure. The second part is the actual result that was computed.

```
def divide(a, b):
```

```

try:
    return True, a / b
except ZeroDivisionError:
    return False, None

```

Callers of this function have to unpack the tuple. That forces them to consider the status part of the tuple instead of just looking at the result of division.

[Click here to view code image](#)

```

success, result = divide(x, y)
if not success:
    print('Invalid inputs')

```

The problem is that callers can easily ignore the first part of the tuple (using the underscore variable name, a Python convention for unused variables). The resulting code doesn't look wrong at first glance. This is as bad as just returning `None`.

```

_, result = divide(x, y)
if not result:
    print('Invalid inputs')

```

The second, better way to reduce these errors is to never return `None` at all. Instead, raise an exception up to the caller and make them deal with it. Here, I turn a `ZeroDivisionError` into a `ValueError` to indicate to the caller that the input values are bad:

[Click here to view code image](#)

```

def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError as e:
        raise ValueError('Invalid inputs') from e

```

Now the caller should handle the exception for the invalid input case (this behavior should be documented; see [Item 49](#): “[Write Docstrings for Every Function, Class, and Module](#)”). The caller no longer requires a condition on the return value of the function. If the function didn't raise an exception, then the return value must be good. The outcome of exception handling is clear.

[Click here to view code image](#)

```

x, y = 5, 2
try:
    result = divide(x, y)
except ValueError:
    print('Invalid inputs')
else:
    print('Result is %.1f' % result)

>>>
Result is 2.5

```

Things to Remember

- ◆ Functions that return `None` to indicate special meaning are error prone because `None` and other values (e.g., zero, the empty string) all evaluate to `False` in conditional expressions.

- ◆ Raise exceptions to indicate special situations instead of returning `None`. Expect the calling code to handle exceptions properly when they're documented.

Item 15: Know How Closures Interact with Variable Scope

Say you want to sort a list of numbers but prioritize one group of numbers to come first. This pattern is useful when you're rendering a user interface and want important messages or exceptional events to be displayed before everything else.

A common way to do this is to pass a helper function as the `key` argument to a list's `sort` method. The helper's return value will be used as the value for sorting each item in the list. The helper can check whether the given item is in the important group and can vary the sort key accordingly.

[Click here to view code image](#)

```
def sort_priority(values, group):
    def helper(x):
        if x in group:
            return (0, x)
        return (1, x)
    values.sort(key=helper)
```

This function works for simple inputs.

[Click here to view code image](#)

```
numbers = [8, 3, 1, 2, 5, 4, 7, 6]
group = {2, 3, 5, 7}
sort_priority(numbers, group)
print(numbers)

>>>
[2, 3, 5, 7, 1, 4, 6, 8]
```

There are three reasons why this function operates as expected:

- Python supports *closures*: functions that refer to variables from the scope in which they were defined. This is why the `helper` function is able to access the `group` argument to `sort_priority`.
- Functions are *first-class* objects in Python, meaning you can refer to them directly, assign them to variables, pass them as arguments to other functions, compare them in expressions and `if` statements, etc. This is how the `sort` method can accept a closure function as the `key` argument.
- Python has specific rules for comparing tuples. It first compares items in index zero, then index one, then index two, and so on. This is why the return value from the `helper` closure causes the sort order to have two distinct groups.

It'd be nice if this function returned whether higher-priority items were seen at all so the user interface code can act accordingly. Adding such behavior seems straightforward. There's already a closure function for deciding which group each number is in. Why not also use the closure to flip a flag when high-priority items are seen? Then the function can return the flag value after it's been modified by the closure.

Here, I try to do that in a seemingly obvious way:

[Click here to view code image](#)

```
def sort_priority2(numbers, group):
    found = False
    def helper(x):
        if x in group:
            found = True # Seems simple
            return (0, x)
        return (1, x)
    numbers.sort(key=helper)
    return found
```

I can run the function on the same inputs as before.

[Click here to view code image](#)

```
found = sort_priority2(numbers, group)
print('Found:', found)
print(numbers)

>>>
Found: False
[2, 3, 5, 7, 1, 4, 6, 8]
```

The sorted results are correct, but the `found` result is wrong. Items from `group` were definitely found in `numbers`, but the function returned `False`. How could this happen?

When you reference a variable in an expression, the Python interpreter will traverse the scope to resolve the reference in this order:

1. The current function's scope
2. Any enclosing scopes (like other containing functions)
3. The scope of the module that contains the code (also called the *global scope*)
4. The built-in scope (that contains functions like `len` and `str`)

If none of these places have a defined variable with the referenced name, then a `NameError` exception is raised.

Assigning a value to a variable works differently. If the variable is already defined in the current scope, then it will just take on the new value. If the variable doesn't exist in the current scope, then Python treats the assignment as a variable definition. The scope of the newly defined variable is the function that contains the assignment.

This assignment behavior explains the wrong return value of the `sort_priority2` function. The `found` variable is assigned to `True` in the `helper` closure. The closure's assignment is treated as a new variable definition within `helper`, not as an assignment within `sort_priority2`.

[Click here to view code image](#)

```
def sort_priority2(numbers, group):
    found = False # Scope: 'sort_priority2'
    def helper(x):
        if x in group:
            found = True # Scope: 'helper' - Bad!
            return (0, x)
```

```
        return (1, x)
    numbers.sort(key=helper)
    return found
```

Encountering this problem is sometimes called the *scoping bug* because it can be so surprising to newbies. But this is the intended result. This behavior prevents local variables in a function from polluting the containing module. Otherwise, every assignment within a function would put garbage into the global module scope. Not only would that be noise, but the interplay of the resulting global variables could cause obscure bugs.

Getting Data Out

In Python 3, there is special syntax for getting data out of a closure. The `nonlocal` statement is used to indicate that scope traversal should happen upon assignment for a specific variable name. The only limit is that `nonlocal` won't traverse up to the module-level scope (to avoid polluting globals).

Here, I define the same function again using `nonlocal`:

[Click here to view code image](#)

```
def sort_priority3(numbers, group):
    found = False
    def helper(x):
        nonlocal found
        if x in group:
            found = True
            return (0, x)
        return (1, x)
    numbers.sort(key=helper)
    return found
```

The `nonlocal` statement makes it clear when data is being assigned out of a closure into another scope. It's complementary to the `global` statement, which indicates that a variable's assignment should go directly into the module scope.

However, much like the anti-pattern of global variables, I'd caution against using `nonlocal` for anything beyond simple functions. The side effects of `nonlocal` can be hard to follow. It's especially hard to understand in long functions where the `nonlocal` statements and assignments to associated variables are far apart.

When your usage of `nonlocal` starts getting complicated, it's better to wrap your state in a helper class. Here, I define a class that achieves the same result as the `nonlocal` approach. It's a little longer, but is much easier to read (see [Item 23: "Accept Functions for Simple Interfaces Instead of Classes"](#) for details on the `__call__` special method).

[Click here to view code image](#)

```
class Sorter(object):
    def __init__(self, group):
        self.group = group
        self.found = False

    def __call__(self, x):
        if x in self.group:
            self.found = True
```

```

        return (0, x)
    return (1, x)

sorter = Sorter(group)
numbers.sort(key=sorter)
assert sorter.found is True

```

Scope in Python 2

Unfortunately, Python 2 doesn't support the `nonlocal` keyword. In order to get similar behavior, you need to use a work-around that takes advantage of Python's scoping rules. This approach isn't pretty, but it's the common Python idiom.

[Click here to view code image](#)

```

# Python 2
def sort_priority(numbers, group):
    found = [False]
    def helper(x):
        if x in group:
            found[0] = True
            return (0, x)
        return (1, x)
    numbers.sort(key=helper)
    return found[0]

```

As explained above, Python will traverse up the scope where the `found` variable is referenced to resolve its current value. The trick is that the value for `found` is a list, which is mutable. This means that once retrieved, the closure can modify the state of `found` to send data out of the inner scope (with `found[0] = True`).

This approach also works when the variable used to traverse the scope is a dictionary, a set, or an instance of a class you've defined.

Things to Remember

- ◆ Closure functions can refer to variables from any of the scopes in which they were defined.
- ◆ By default, closures can't affect enclosing scopes by assigning variables.
- ◆ In Python 3, use the `nonlocal` statement to indicate when a closure can modify a variable in its enclosing scopes.
- ◆ In Python 2, use a mutable value (like a single-item list) to work around the lack of the `nonlocal` statement.
- ◆ Avoid using `nonlocal` statements for anything beyond simple functions.

Item 16: Consider Generators Instead of Returning Lists

The simplest choice for functions that produce a sequence of results is to return a list of items. For example, say you want to find the index of every word in a string. Here, I accumulate results in a list using the `append` method and return it at the end of the function:

[Click here to view code image](#)

```
def index_words(text):
    result = []
    if text:
        result.append(0)
    for index, letter in enumerate(text):
        if letter == ' ':
            result.append(index + 1)
    return result
```

This works as expected for some sample input.

[Click here to view code image](#)

```
address = 'Four score and seven years ago...'
result = index_words(address)
print(result[:3])

>>>
[0, 5, 11]
```

There are two problems with the `index_words` function.

The first problem is that the code is a bit dense and noisy. Each time a new result is found, I call the `append` method. The method call's bulk (`result.append`) deemphasizes the value being added to the list (`index + 1`). There is one line for creating the result list and another for returning it. While the function body contains ~130 characters (without whitespace), only ~75 characters are important.

A better way to write this function is using a *generator*. Generators are functions that use `yield` expressions. When called, generator functions do not actually run but instead immediately return an iterator. With each call to the `next` built-in function, the iterator will advance the generator to its next `yield` expression. Each value passed to `yield` by the generator will be returned by the iterator to the caller.

Here, I define a generator function that produces the same results as before:

[Click here to view code image](#)

```
def index_words_iter(text):
    if text:
        yield 0
    for index, letter in enumerate(text):
        if letter == ' ':
            yield index + 1
```

It's significantly easier to read because all interactions with the result list have been eliminated. Results are passed to `yield` expressions instead. The iterator returned by the generator call can easily be converted to a list by passing it to the `list` built-in function (see [Item 9: "Consider Generator Expressions for Large Comprehensions"](#) for how this works).

[Click here to view code image](#)

```
result = list(index_words_iter(address))
```

The second problem with `index_words` is that it requires all results to be stored in the list before being returned. For huge inputs, this can cause your program to run out of

memory and crash. In contrast, a generator version of this function can easily be adapted to take inputs of arbitrary length.

Here, I define a generator that streams input from a file one line at a time and yields outputs one word at a time. The working memory for this function is bounded to the maximum length of one line of input.

```
def index_file(handle):
    offset = 0
    for line in handle:
        if line:
            yield offset
            for letter in line:
                offset += 1
                if letter == ' ':
                    yield offset
```

Running the generator produces the same results.

[Click here to view code image](#)

```
with open('/tmp/address.txt', 'r') as f:
    it = index_file(f)
    results = islice(it, 0, 3)
    print(list(results))
```

```
>>>
[0, 5, 11]
```

The only gotcha of defining generators like this is that the callers must be aware that the iterators returned are stateful and can't be reused (see [Item 17: “Be Defensive When Iterating Over Arguments”](#)).

Things to Remember

- ◆ Using generators can be clearer than the alternative of returning lists of accumulated results.
- ◆ The iterator returned by a generator produces the set of values passed to `yield` expressions within the generator function's body.
- ◆ Generators can produce a sequence of outputs for arbitrarily large inputs because their working memory doesn't include all inputs and outputs.

Item 17: Be Defensive When Iterating Over Arguments

When a function takes a list of objects as a parameter, it's often important to iterate over that list multiple times. For example, say you want to analyze tourism numbers for the U.S. state of Texas. Imagine the data set is the number of visitors to each city (in millions per year). You'd like to figure out what percentage of overall tourism each city receives.

To do this you need a normalization function. It sums the inputs to determine the total number of tourists per year. Then it divides each city's individual visitor count by the total to find that city's contribution to the whole.

[Click here to view code image](#)


```
def normalize(numbers):
    total = sum(numbers)
    result = []
    for value in numbers:
        percent = 100 * value / total
        result.append(percent)
    return result
```

This function works when given a list of visits.

[Click here to view code image](#)

```
visits = [15, 35, 80]
percentages = normalize(visits)
print(percentages)

>>>
[11.538461538461538, 26.923076923076923, 61.53846153846154]
```

To scale this up, I need to read the data from a file that contains every city in all of Texas. I define a generator to do this because then I can reuse the same function later when I want to compute tourism numbers for the whole world, a much larger data set (see [Item 16: “Consider Generators Instead of Returning Lists”](#)).

[Click here to view code image](#)

```
def read_visits(data_path):
    with open(data_path) as f:
        for line in f:
            yield int(line)
```

Surprisingly, calling `normalize` on the generator’s return value produces no results.

[Click here to view code image](#)

```
it = read_visits('/tmp/my_numbers.txt')
percentages = normalize(it)
print(percentages)

>>>
[]
```

The cause of this behavior is that an iterator only produces its results a single time. If you iterate over an iterator or generator that has already raised a `StopIteration` exception, you won’t get any results the second time around.

[Click here to view code image](#)

```
it = read_visits('/tmp/my_numbers.txt')
print(list(it))
print(list(it)) # Already exhausted

>>>
[15, 35, 80]
[]
```

What’s confusing is that you also won’t get any errors when you iterate over an already exhausted iterator. `for` loops, the `list` constructor, and many other functions throughout the Python standard library expect the `StopIteration` exception to be raised during normal operation. These functions can’t tell the difference between an iterator that has no output and an iterator that had output and is now exhausted.

To solve this problem, you can explicitly exhaust an input iterator and keep a copy of its entire contents in a list. You can then iterate over the list version of the data as many times as you need to. Here's the same function as before, but it defensively copies the input iterator:

[Click here to view code image](#)

```
def normalize_copy(numbers):
    numbers = list(numbers) # Copy the iterator
    total = sum(numbers)
    result = []
    for value in numbers:
        percent = 100 * value / total
        result.append(percent)
    return result
```

Now the function works correctly on a generator's return value.

[Click here to view code image](#)

```
it = read_visits('/tmp/my_numbers.txt')
percentages = normalize_copy(it)
print(percentages)

>>>
[11.538461538461538, 26.923076923076923, 61.53846153846154]
```

The problem with this approach is the copy of the input iterator's contents could be large. Copying the iterator could cause your program to run out of memory and crash. One way around this is to accept a function that returns a new iterator each time it's called.

[Click here to view code image](#)

```
def normalize_func(get_iter):
    total = sum(get_iter()) # New iterator
    result = []
    for value in get_iter(): # New iterator
        percent = 100 * value / total
        result.append(percent)
    return result
```

To use `normalize_func`, you can pass in a `lambda` expression that calls the generator and produces a new iterator each time.

[Click here to view code image](#)

```
percentages = normalize_func(lambda: read_visits(path))
```

Though it works, having to pass a `lambda` function like this is clumsy. The better way to achieve the same result is to provide a new container class that implements the *iterator protocol*.

The iterator protocol is how Python `for` loops and related expressions traverse the contents of a container type. When Python sees a statement like `for x in foo` it will actually call `iter(foo)`. The `iter` built-in function calls the `foo.__iter__` special method in turn. The `__iter__` method must return an iterator object (which itself implements the `__next__` special method). Then the `for` loop repeatedly calls the `next` built-in function on the iterator object until it's exhausted (and raises a `StopIteration` exception).

It sounds complicated, but practically speaking you can achieve all of this behavior for your classes by implementing the `__iter__` method as a generator. Here, I define an iterable container class that reads the files containing tourism data:

[Click here to view code image](#)

```
class ReadVisits(object):
    def __init__(self, data_path):
        self.data_path = data_path

    def __iter__(self):
        with open(self.data_path) as f:
            for line in f:
                yield int(line)
```

This new container type works correctly when passed to the original function without any modifications.

[Click here to view code image](#)

```
visits = ReadVisits(path)
percentages = normalize(visits)
print(percentages)

>>>
[11.538461538461538, 26.923076923076923, 61.53846153846154]
```

This works because the `sum` method in `normalize` will call `ReadVisits.__iter__` to allocate a new iterator object. The `for` loop to `normalize` the numbers will also call `__iter__` to allocate a second iterator object. Each of those iterators will be advanced and exhausted independently, ensuring that each unique iteration sees all of the input data values. The only downside of this approach is that it reads the input data multiple times.

Now that you know how containers like `ReadVisits` work, you can write your functions to ensure that parameters aren't just iterators. The protocol states that when an iterator is passed to the `iter` built-in function, `iter` will return the iterator itself. In contrast, when a container type is passed to `iter`, a new iterator object will be returned each time. Thus, you can test an input value for this behavior and raise a `TypeError` to reject iterators.

[Click here to view code image](#)

```
def normalize_defensive(numbers):
    if iter(numbers) is iter(numbers): # An iterator - bad!
        raise TypeError('Must supply a container')
    total = sum(numbers)
    result = []
    for value in numbers:
        percent = 100 * value / total
        result.append(percent)
    return result
```

This is ideal if you don't want to copy the full input iterator like `normalize_copy` above, but you also need to iterate over the input data multiple times. This function works as expected for `list` and `ReadVisits` inputs because they are containers. It will work for any type of container that follows the iterator protocol.

[Click here to view code image](#)

```
visits = [15, 35, 80]
normalize_defensive(visits) # No error
visits = ReadVisits(path)
normalize_defensive(visits) # No error
```

The function will raise an exception if the input is iterable but not a container.

[Click here to view code image](#)

```
it = iter(visits)
normalize_defensive(it)

>>>
TypeError: Must supply a container
```

Things to Remember

- ◆ Beware of functions that iterate over input arguments multiple times. If these arguments are iterators, you may see strange behavior and missing values.
- ◆ Python's iterator protocol defines how containers and iterators interact with the `iter` and `next` built-in functions, `for` loops, and related expressions.
- ◆ You can easily define your own iterable container type by implementing the `__iter__` method as a generator.
- ◆ You can detect that a value is an iterator (instead of a container) if calling `iter` on it twice produces the same result, which can then be progressed with the `next` built-in function.

Item 18: Reduce Visual Noise with Variable Positional Arguments

Accepting optional positional arguments (often called *star args* in reference to the conventional name for the parameter, `*args`) can make a function call more clear and remove *visual noise*.

For example, say you want to log some debug information. With a fixed number of arguments, you would need a function that takes a message and a list of values.

[Click here to view code image](#)

```
def log(message, values):
    if not values:
        print(message)
    else:
        values_str = ', '.join(str(x) for x in values)
        print('%s: %s' % (message, values_str))

log('My numbers are', [1, 2])
log('Hi there', [])

>>>
My numbers are: 1, 2
Hi there
```

Having to pass an empty list when you have no values to log is cumbersome and noisy. It'd be better to leave out the second argument entirely. You can do this in Python by prefixing the last positional parameter name with `*`. The first parameter for the log message is required, whereas any number of subsequent positional arguments are optional. The function body doesn't need to change, only the callers do.

[Click here to view code image](#)

```
def log(message, *values): # The only difference
    if not values:
        print(message)
    else:
        values_str = ', '.join(str(x) for x in values)
        print('%s: %s' % (message, values_str))

log('My numbers are', 1, 2)
log('Hi there') # Much better

>>>
My numbers are: 1, 2
Hi there
```

If you already have a list and want to call a variable argument function like `log`, you can do this by using the `*` operator. This instructs Python to pass items from the sequence as positional arguments.

[Click here to view code image](#)

```
favorites = [7, 33, 99]
log('Favorite colors', *favorites)

>>>
Favorite colors: 7, 33, 99
```

There are two problems with accepting a variable number of positional arguments.

The first issue is that the variable arguments are always turned into a tuple before they are passed to your function. This means that if the caller of your function uses the `*` operator on a generator, it will be iterated until it's exhausted. The resulting tuple will include every value from the generator, which could consume a lot of memory and cause your program to crash.

[Click here to view code image](#)

```
def my_generator():
    for i in range(10):
        yield i

def my_func(*args):
    print(args)

it = my_generator()
my_func(*it)

>>>
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

Functions that accept `*args` are best for situations where you know the number of inputs in the argument list will be reasonably small. It's ideal for function calls that pass many

literals or variable names together. It's primarily for the convenience of the programmer and the readability of the code.

The second issue with `*args` is that you can't add new positional arguments to your function in the future without migrating every caller. If you try to add a positional argument in the front of the argument list, existing callers will subtly break if they aren't updated.

[Click here to view code image](#)

```
def log(sequence, message, *values):
    if not values:
        print('%s: %s' % (sequence, message))
    else:
        values_str = ', '.join(str(x) for x in values)
        print('%s: %s: %s' % (sequence, message, values_str))

log(1, 'Favorites', 7, 33)      # New usage is OK
log('Favorite numbers', 7, 33) # Old usage breaks

>>>
1: Favorites: 7, 33
Favorite numbers: 7: 33
```

The problem here is that the second call to `log` used 7 as the `message` parameter because a `sequence` argument wasn't given. Bugs like this are hard to track down because the code still runs without raising any exceptions. To avoid this possibility entirely, you should use keyword-only arguments when you want to extend functions that accept `*args` (see [Item 21: “Enforce Clarity with Keyword-Only Arguments”](#)).

Things to Remember

- ◆ Functions can accept a variable number of positional arguments by using `*args` in the `def` statement.
- ◆ You can use the items from a sequence as the positional arguments for a function with the `*` operator.
- ◆ Using the `*` operator with a generator may cause your program to run out of memory and crash.
- ◆ Adding new positional parameters to functions that accept `*args` can introduce hard-to-find bugs.

Item 19: Provide Optional Behavior with Keyword Arguments

Like most other programming languages, calling a function in Python allows for passing arguments by position.

[Click here to view code image](#)

```
def remainder(number, divisor):
    return number % divisor
```

```
assert remainder(20, 7) == 6
```

All positional arguments to Python functions can also be passed by keyword, where the name of the argument is used in an assignment within the parentheses of a function call. The keyword arguments can be passed in any order as long as all of the required positional arguments are specified. You can mix and match keyword and positional arguments. These calls are equivalent:

[Click here to view code image](#)

```
remainder(20, 7)
remainder(20, divisor=7)
remainder(number=20, divisor=7)
remainder(divisor=7, number=20)
```

Positional arguments must be specified before keyword arguments.

[Click here to view code image](#)

```
remainder(number=20, 7)

>>>
SyntaxError: non-keyword arg after keyword arg
```

Each argument can only be specified once.

[Click here to view code image](#)

```
remainder(20, number=7)

>>>
TypeError: remainder() got multiple values for argument 'number'
```

The flexibility of keyword arguments provides three significant benefits.

The first advantage is that keyword arguments make the function call clearer to new readers of the code. With the call `remainder(20, 7)`, it's not evident which argument is the number and which is the divisor without looking at the implementation of the `remainder` method. In the call with keyword arguments, `number=20` and `divisor=7` make it immediately obvious which parameter is being used for each purpose.

The second impact of keyword arguments is that they can have default values specified in the function definition. This allows a function to provide additional capabilities when you need them but lets you accept the default behavior most of the time. This can eliminate repetitive code and reduce noise.

For example, say you want to compute the rate of fluid flowing into a vat. If the vat is also on a scale, then you could use the difference between two weight measurements at two different times to determine the flow rate.

[Click here to view code image](#)

```
def flow_rate(weight_diff, time_diff):
    return weight_diff / time_diff

weight_diff = 0.5
time_diff = 3
flow = flow_rate(weight_diff, time_diff)
print('%.3f kg per second' % flow)
```


The only problem with this approach is that optional keyword arguments like `period` and `units_per_kg` may still be specified as positional arguments.

[Click here to view code image](#)

```
pounds_per_hour = flow_rate(weight_diff, time_diff, 3600, 2.2)
```

Supplying optional arguments positionally can be confusing because it isn't clear what the values `3600` and `2.2` correspond to. The best practice is to always specify optional arguments using the keyword names and never pass them as positional arguments.

Note

Backwards compatibility using optional keyword arguments like this is crucial for functions that accept `*args` (see [Item 18: “Reduce Visual Noise with Variable Positional Arguments”](#)). But an even better practice is to use keyword-only arguments (see [Item 21: “Enforce Clarity with Keyword-Only Arguments”](#)).

Things to Remember

- ◆ Function arguments can be specified by position or by keyword.
- ◆ Keywords make it clear what the purpose of each argument is when it would be confusing with only positional arguments.
- ◆ Keyword arguments with default values make it easy to add new behaviors to a function, especially when the function has existing callers.
- ◆ Optional keyword arguments should always be passed by keyword instead of by position.

Item 20: Use `None` and Docstrings to Specify Dynamic Default Arguments

Sometimes you need to use a non-static type as a keyword argument's default value. For example, say you want to print logging messages that are marked with the time of the logged event. In the default case, you want the message to include the time when the function was called. You might try the following approach, assuming the default arguments are reevaluated each time the function is called.

[Click here to view code image](#)

```
def log(message, when=datetime.now()):
    print('%s: %s' % (when, message))

log('Hi there!')
sleep(0.1)
log('Hi again!')
```

```
>>>
2014-11-15 21:10:10.371432: Hi there!
2014-11-15 21:10:10.371432: Hi again!
```

The timestamps are the same because `datetime.now` is only executed a single time:

when the function is defined. Default argument values are evaluated only once per module load, which usually happens when a program starts up. After the module containing this code is loaded, the `datetime.now` default argument will never be evaluated again.

The convention for achieving the desired result in Python is to provide a default value of `None` and to document the actual behavior in the docstring (see [Item 49: “Write Docstrings for Every Function, Class, and Module”](#)). When your code sees an argument value of `None`, you allocate the default value accordingly.

[Click here to view code image](#)

```
def log(message, when=None):
    """Log a message with a timestamp.

    Args:
        message: Message to print.
        when: datetime of when the message occurred.
            Defaults to the present time.
    """
    when = datetime.now() if when is None else when
    print('%s: %s' % (when, message))
```

Now the timestamps will be different.

[Click here to view code image](#)

```
log('Hi there!')
sleep(0.1)
log('Hi again!')
```

```
>>>
2014-11-15 21:10:10.472303: Hi there!
2014-11-15 21:10:10.573395: Hi again!
```

Using `None` for default argument values is especially important when the arguments are mutable. For example, say you want to load a value encoded as JSON data. If decoding the data fails, you want an empty dictionary to be returned by default. You might try this approach.

[Click here to view code image](#)

```
def decode(data, default={}):
    try:
        return json.loads(data)
    except ValueError:
        return default
```

The problem here is the same as the `datetime.now` example above. The dictionary specified for `default` will be shared by all calls to `decode` because default argument values are only evaluated once (at module load time). This can cause extremely surprising behavior.

```
foo = decode('bad data')
foo['stuff'] = 5
bar = decode('also bad')
bar['meep'] = 1
print('Foo:', foo)
print('Bar:', bar)
```

```
>>>
```

```
Foo: {'stuff': 5, 'meep': 1}
Bar: {'stuff': 5, 'meep': 1}
```

You'd expect two different dictionaries, each with a single key and value. But modifying one seems to also modify the other. The culprit is that `foo` and `bar` are both equal to the `default` parameter. They are the same dictionary object.

```
assert foo is bar
```

The fix is to set the keyword argument default value to `None` and then document the behavior in the function's docstring.

[Click here to view code image](#)

```
def decode(data, default=None):
    """Load JSON data from a string.

    Args:
        data: JSON data to decode.
        default: Value to return if decoding fails.
                Defaults to an empty dictionary.
    """
    if default is None:
        default = {}
    try:
        return json.loads(data)
    except ValueError:
        return default
```

Now, running the same test code as before produces the expected result.

```
foo = decode('bad data')
foo['stuff'] = 5
bar = decode('also bad')
bar['meep'] = 1
print('Foo:', foo)
print('Bar:', bar)

>>>
Foo: {'stuff': 5}
Bar: {'meep': 1}
```

Things to Remember

- ◆ Default arguments are only evaluated once: during function definition at module load time. This can cause odd behaviors for dynamic values (like `{}` or `[]`).
- ◆ Use `None` as the default value for keyword arguments that have a dynamic value. Document the actual default behavior in the function's docstring.

Item 21: Enforce Clarity with Keyword-Only Arguments

Passing arguments by keyword is a powerful feature of Python functions (see [Item 19: “Provide Optional Behavior with Keyword Arguments”](#)). The flexibility of keyword arguments enables you to write code that will be clear for your use cases.

For example, say you want to divide one number by another but be very careful about special cases. Sometimes you want to ignore `ZeroDivisionError` exceptions and

return infinity instead. Other times, you want to ignore `OverflowError` exceptions and return zero instead.

[Click here to view code image](#)

```
def safe_division(number, divisor, ignore_overflow,
                  ignore_zero_division):
    try:
        return number / divisor
    except OverflowError:
        if ignore_overflow:
            return 0
        else:
            raise
    except ZeroDivisionError:
        if ignore_zero_division:
            return float('inf')
        else:
            raise
```

Using this function is straightforward. This call will ignore the `float` overflow from division and will return zero.

[Click here to view code image](#)

```
result = safe_division(1, 10**500, True, False)
print(result)

>>>
0.0
```

This call will ignore the error from dividing by zero and will return infinity.

[Click here to view code image](#)

```
result = safe_division(1, 0, False, True)
print(result)

>>>
inf
```

The problem is that it's easy to confuse the position of the two Boolean arguments that control the exception-ignoring behavior. This can easily cause bugs that are hard to track down. One way to improve the readability of this code is to use keyword arguments. By default, the function can be overly cautious and can always re-raise exceptions.

[Click here to view code image](#)

```
def safe_division_b(number, divisor,
                    ignore_overflow=False,
                    ignore_zero_division=False):
    # ...
```

Then callers can use keyword arguments to specify which of the ignore flags they want to flip for specific operations, overriding the default behavior.

[Click here to view code image](#)

```
safe_division_b(1, 10**500, ignore_overflow=True)
safe_division_b(1, 0, ignore_zero_division=True)
```

The problem is, since these keyword arguments are optional behavior, there's nothing forcing callers of your functions to use keyword arguments for clarity. Even with the new

definition of `safe_division_b`, you can still call it the old way with positional arguments.

[Click here to view code image](#)

```
safe_division_b(1, 10**500, True, False)
```

With complex functions like this, it's better to require that callers are clear about their intentions. In Python 3, you can demand clarity by defining your functions with keyword-only arguments. These arguments can only be supplied by keyword, never by position.

Here, I redefine the `safe_division` function to accept keyword-only arguments. The `*` symbol in the argument list indicates the end of positional arguments and the beginning of keyword-only arguments.

[Click here to view code image](#)

```
def safe_division_c(number, divisor, *,
                    ignore_overflow=False,
                    ignore_zero_division=False):
    # ...
```

Now, calling the function with positional arguments for the keyword arguments won't work.

[Click here to view code image](#)

```
safe_division_c(1, 10**500, True, False)
```

```
>>>
```

```
TypeError: safe_division_c() takes 2 positional arguments but 4 were given
```

Keyword arguments and their default values work as expected.

[Click here to view code image](#)

```
safe_division_c(1, 0, ignore_zero_division=True) # OK

try:
    safe_division_c(1, 0)
except ZeroDivisionError:
    pass # Expected
```

Keyword-Only Arguments in Python 2

Unfortunately, Python 2 doesn't have explicit syntax for specifying keyword-only arguments like Python 3. But you can achieve the same behavior of raising `TypeError`s for invalid function calls by using the `**` operator in argument lists. The `**` operator is similar to the `*` operator (see [Item 18: "Reduce Visual Noise with Variable Positional Arguments"](#)), except that instead of accepting a variable number of positional arguments, it accepts any number of keyword arguments, even when they're not defined.

[Click here to view code image](#)

```
# Python 2
def print_args(*args, **kwargs):
    print 'Positional:', args
    print 'Keyword:   ', kwargs

print_args(1, 2, foo='bar', stuff='meep')
```

```
>>>
Positional: (1, 2)
Keyword:    {'foo': 'bar', 'stuff': 'meep'}
```

To make `safe_division` take keyword-only arguments in Python 2, you have the function accept `**kwargs`. Then you `pop` keyword arguments that you expect out of the `kwargs` dictionary, using the `pop` method's second argument to specify the default value when the key is missing. Finally, you make sure there are no more keyword arguments left in `kwargs` to prevent callers from supplying arguments that are invalid.

[Click here to view code image](#)

```
# Python 2
def safe_division_d(number, divisor, **kwargs):
    ignore_overflow = kwargs.pop('ignore_overflow', False)
    ignore_zero_div = kwargs.pop('ignore_zero_division', False)
    if kwargs:
        raise TypeError('Unexpected **kwargs: %r' % kwargs)
    # ...
```

Now, you can call the function with or without keyword arguments.

[Click here to view code image](#)

```
safe_division_d(1, 10)
safe_division_d(1, 0, ignore_zero_division=True)
safe_division_d(1, 10**500, ignore_overflow=True)
```

Trying to pass keyword-only arguments by position won't work, just like in Python 3.

[Click here to view code image](#)

```
safe_division_d(1, 0, False, True)
```

```
>>>
TypeError: safe_division_d() takes 2 positional arguments but 4 were given
```

Trying to pass unexpected keyword arguments also won't work.

[Click here to view code image](#)

```
safe_division_d(0, 0, unexpected=True)
```

```
>>>
TypeError: Unexpected **kwargs: {'unexpected': True}
```

Things to Remember

- ◆ Keyword arguments make the intention of a function call more clear.
- ◆ Use keyword-only arguments to force callers to supply keyword arguments for potentially confusing functions, especially those that accept multiple Boolean flags.
- ◆ Python 3 supports explicit syntax for keyword-only arguments in functions.
- ◆ Python 2 can emulate keyword-only arguments for functions by using `**kwargs` and manually raising `TypeError` exceptions.